



Detecção de Eventos em Séries Temporais

parte 6 - Pontos de mudança, online e drift



Eduardo Ogasawara

eogasawara@cefet-rj.br | <https://eic.cefet-rj.br/~eogasawara>



Razão de Verossimilhança

Ideia Central

Compara quão mais provável é a série com dois modelos distintos do que com um único: se a melhoria for grande, há mudança estrutural.

$$\Lambda(\tau) = 2 \left[\ell(\hat{\theta}_1, \hat{\theta}_2 \mid \tau) - \ell(\hat{\theta} \mid \text{sem quebra}) \right]$$

Sob H_0 , $\Lambda(\tau) \sim \chi^2(p)$ assintoticamente. Para τ desconhecido, usa-se $\sup_{\tau} \Lambda(\tau)$, com correção do p-valor pela cota de Davies.

ENTRADA: série x , família de modelos, candidato τ (ou grade de candidatos)

PRÉ-CONDIÇÃO: modelo identificável; amostra suficiente em cada segmento

PASSO 1 — Ajuste do modelo irrestrito

Estimar $\hat{\theta}$ na série completa $\rightarrow \ell(\hat{\theta})$

PASSO 2 — Ajuste do modelo com quebra em τ

Estimar $\hat{\theta}_1$ em $x_{\{1:\tau\}}$ e $\hat{\theta}_2$ em $x_{\{\tau+1:T\}}$

Calcular $\ell(\hat{\theta}_1, \hat{\theta}_2 \mid \tau)$

PASSO 3 — Estatística e decisão

$\Lambda(\tau) \leftarrow 2 \cdot [\ell(\hat{\theta}_1, \hat{\theta}_2 \mid \tau) - \ell(\hat{\theta})]$

SE τ fixo: comparar $\Lambda(\tau)$ com $\chi^2_{\alpha}(p)$

SE τ livre: $\tau^* \leftarrow \operatorname{argmax} \Lambda(\tau)$; aplicar cota de Davies

SAÍDA: τ^* e decisão sobre quebra estrutural

PÓS-CONDIÇÃO: generaliza o Teste de Chow para qualquer família de modelos

Programação Dinâmica para Segmentação

Ideia Central

Encontra a partição globalmente ótima minimizando custo de ajuste penalizado pelo número de cortes. β controla o trade-off entre ajuste e complexidade: BIC sugere $\beta = p \cdot \log T$, e PELT reduz a complexidade de $\mathcal{O}(T^2)$ para $\mathcal{O}(T)$ via poda.

$$V(t) = \min_{1 \leq s < t} [V(s) + \mathcal{C}(s+1, t) + \beta]$$

ENTRADA: série x , função de custo $\mathcal{C}(\cdot, \cdot)$, penalização β

PRÉ-CONDIÇÃO: $\mathcal{C}(s, t)$ computável em $\mathcal{O}(1)$ após pré-processamento

PASSO 1 — Inicialização

$V(0) \leftarrow 0$; $cp(0) \leftarrow []$

PASSO 2 — Recursão (PELT)

PARA t de 1 até T :

 candidatos $\leftarrow \{s \in \text{admissíveis} : s < t\}$

 PARA cada s em candidatos:

$\text{custo}(s) \leftarrow V(s) + \mathcal{C}(s+1, t) + \beta$

$s^* \leftarrow \text{argmin } \text{custo}(s)$

$V(t) \leftarrow \text{custo}(s^*)$

$cp(t) \leftarrow cp(s^*) \cup \{s^*\}$

 PODAR candidatos segundo a regra de PELT

PASSO 3 — Recuperação dos pontos

RETORNAR $cp(T)$

SAÍDA: conjunto ótimo de pontos de mudança

PÓS-CONDIÇÃO: solução globalmente ótima para o β dado

Change Points via HMM

Ideia Central

Modela a série como alternância entre K regimes ocultos. Uma mudança estrutural é uma transição de estado latente z_t .

$$P(z_t \mid z_{t-1}) = A_{z_{t-1}, z_t}$$

$$P(x_t \mid z_t) = \mathcal{N}(x_t; \mu_{z_t}, \Sigma_{z_t})$$

Viterbi recupera a sequência de estados mais provável; Baum-Welch (EM) estima A, μ, Σ . Transições $z_t^* \neq z_{t-1}^*$ indicam mudança de regime. O HMM é adequado para séries com regimes recorrentes, pois o mesmo estado pode ser revisitado.

ENTRADA: série x , número de estados K

PRÉ-CONDIÇÃO: K definido a priori; parâmetros inicializados (e.g., K-means)

// BAUM-WELCH (estimação de parâmetros)

PASSO 1 — E-step: Forward-Backward

PARA t de 1 até T :

$\alpha_t(k) \leftarrow P(x_{1:t}, z_t=k)$ // forward

$\beta_t(k) \leftarrow P(x_{t+1:T} \mid z_t=k)$ // backward

$\gamma_t(k) \leftarrow \alpha_t(k) \cdot \beta_t(k) / P(x)$ // suavização

PASSO 2 — M-step: atualização

$A_{ij} \leftarrow \sum_t \xi_t(i,j) / \sum_t \gamma_t(i)$

$\mu_k \leftarrow \sum_t \gamma_t(k) \cdot x_t / \sum_t \gamma_t(k)$

REPETIR até convergência

// VITERBI (decodificação)

PASSO 3 — Sequência ótima

$z^* \leftarrow \operatorname{argmax}_z P(z \mid x, A, \mu, \Sigma)$

SAÍDA: sequência z^* ; pontos de mudança onde $z_t^* \neq z_{t-1}^*$

PÓS-CONDIÇÃO: mesmo regime pode reaparecer ao longo da série

Change Points via Aprendizado de Máquina

Ideia Central

Treina um modelo no comportamento típico da série. Quando o erro de previsão agregado em janela aumenta persistentemente, há mudança de regime.

$$S_t = \frac{1}{w} \sum_{i=t-w+1}^t \|x_i - f(x_{i-1}, \dots, x_{i-p})\|$$

S_t excedendo o limiar h sinaliza mudança. O score em janela evita confundir anomalias pontuais com rupturas estruturais.

ENTRADA: série x , modelo f , tamanho de janela w , limiar h

PRÉ-CONDIÇÃO: f treinado em janela de referência estável

PASSO 1 — Estimação do limiar de referência

Calcular erros $e_i = \|x_i - f(x_{i-1:p})\|$ na janela de referência

$h \leftarrow$ percentil alto da distribuição de $\{e_i\}$

PASSO 2 — Monitoramento online

PARA t de w até T :

$S_t \leftarrow$ média de $e_{t-w+1}, \dots, e_t$

PASSO 3 — Regra de decisão

SE $S_t > h$:

SINALIZAR mudança em t

SAÍDA: instantes t onde mudança foi sinalizada

PÓS-CONDIÇÃO: w controla sensibilidade — janelas maiores reduzem falsos positivos mas aumentam atraso

Detecção Temporal: Modos de Operação

A detecção de mudanças pode ocorrer em dois paradigmas fundamentais, definidos pelo acesso — ou não — a observações futuras.

Detecção Offline

Toda a série temporal está disponível de uma vez. O algoritmo pode **olhar para frente**, utilizando observações futuras para identificar mudanças passadas. Adequada para análises retrospectivas com divisão clássica em treino e teste.

$$X = (x_1, x_2, \dots, x_T)$$

A série completa com T observações está disponível para análise global.

Detecção Online

As observações chegam sequencialmente. O algoritmo **não pode olhar para frente** — apenas o passado e o presente são conhecidos. Há um **tradeoff** entre rapidez na detecção e taxa de falsos positivos. O lote i é usado para prever o lote $i+1$.

$$X_t = (x_1, x_2, \dots, x_t)$$

A cada instante t , apenas as observações acumuladas até o presente estão disponíveis.

Latência de Detecção

Evento Real

Ocorre no instante τ

Detecção

Identificado em $\hat{\tau}$

Atraso

Latência de k observações

A latência de detecção é inerente ao paradigma online. Como os algoritmos precisam **inspecionar observações recentes acumuladas** antes de determinar se um evento ocorreu, há sempre um atraso entre o instante real τ e o instante de identificação $\hat{\tau}$.

$$\Delta t = \hat{\tau} - \tau \geq k$$

A classificação precisa só é possível após uma janela mínima de k observações — exigindo equilíbrio entre **urgência na detecção** e **acurácia na classificação**, tensão crítica em séries com *concept drift* ou padrões evolutivos.

Restrições do Paradigma Online

Memória Limitada

Armazenar toda a série histórica é inviável; apenas uma fração dos dados é mantida.

Tempo de Processamento

Cada observação deve ser processada à medida que chega — fluxos rápidos mudam em segundos.

Sem Supervisão

Sem divisão estática de treino/teste nem acesso a observações futuras; o algoritmo opera sem supervisão ou com supervisão parcial.

Estabilidade-Plasticidade

Adaptar-se a mudanças relevantes sem ser perturbado por variações irrelevantes ou ruído transitório.

Essas restrições definem os limites estruturais fundamentais da detecção de mudanças em tempo real.

Janela Deslizante

Em aprendizado de fluxo contínuo, o algoritmo precisa decidir **quanta memória histórica manter**. Existem três estratégias fundamentais:

Memória Total

Considera **todas** as observações acumuladas. Oferece contexto máximo, mas torna-se inviável conforme o fluxo cresce.

Definição Formal

A janela deslizante de tamanho w no instante t é definida como:

$$W_t = (x_{t-w+1}, \dots, x_t)$$

A cada novo dado x_t , a observação mais antiga x_{t-w} é descartada. Janelas maiores capturam mais contexto; janelas menores reagem mais rápido a mudanças.

Sem Memória

Processa apenas o **exemplo atual**. Eficiente, mas incapaz de capturar tendências ao longo do tempo.

Memória em Janela ✓

Mantém apenas as **observações mais recentes**. Equilibra contexto, velocidade e escalabilidade.

Pseudocódigo

ENTRADA: x (série temporal), w (tamanho da janela)

PRÉ-CONDIÇÃO: $w \geq 1, t \geq w$

PASSO 1 — Inicialização

$W \leftarrow (x_1, x_2, \dots, x_w)$

PASSO 2 — Atualização a cada novo x_t

PARA t de $w+1$ até T :

$W \leftarrow (x_{t-w+1}, \dots, x_t)$ descarta x_{t-w} , inclui x_t

estatística $\leftarrow f(W)$ ex: média, variância

SAÍDA: sequência de estatísticas sobre W_t

PÓS-CONDIÇÃO: apenas os w valores mais recentes são retidos

Estatísticas Incrementais

Recalcular estatísticas do zero a cada nova observação é inviável em tempo real. O paradigma incremental atualiza a estimativa usando apenas o valor anterior e a nova observação.

$$\mu_t = \mu_{t-1} + (x_t - \mu_{t-1})/t$$

O termo $(x_t - \mu_{t-1})$ é o erro de predição — a correção aplicada a cada passo. Atualização em tempo $O(1)$, memória constante.

ENTRADA: x (fluxo de observações)

PRÉ-CONDIÇÃO: $t \geq 1$, μ_0 inicializado (ex: $\mu_0 = x_1$)

PASSO 1 — Inicialização

$\mu_0 \leftarrow x_1$

$\sigma^2_0 \leftarrow 0$

PASSO 2 — Para cada nova observação x_t ($t \geq 2$):

$\delta \leftarrow x_t - \mu_{t-1}$ erro de predição

$\mu_t \leftarrow \mu_{t-1} + \delta / t$ média incremental

$\sigma^2_t \leftarrow \sigma^2_{t-1} + \delta \cdot (x_t - \mu_t)$ variância (Welford)

SAÍDA: sequência de estimativas (μ_t, σ^2_t) para cada t

PÓS-CONDIÇÃO: equivalente ao cálculo em lote, sem armazenar histórico

Modelos Adaptativos

Teoria

Em cenários online, os parâmetros do modelo são ajustados incrementalmente a cada nova observação para acompanhar mudanças graduais no processo.

$$\theta_t = \theta_{t-1} + \eta \cdot \nabla L(x_t, \theta_{t-1})$$

Onde η é a taxa de aprendizado e ∇L é o gradiente da função de perda.

Pseudocódigo

ENTRADA: x (fluxo de observações), θ_0 (parâmetros iniciais), η (taxa de aprendizado)

PRÉ-CONDIÇÃO: modelo inicializado com dados históricos ou prior

PASSO 1 — Para cada nova observação x_t :

$s_t \leftarrow \text{score}(x_t, \theta_{t-1})$ desvio em relação ao modelo atual

PASSO 2 — Decisão

SE $s_t > \tau$: emitir alerta de mudança

PASSO 3 — Atualização do modelo

$\theta_t \leftarrow \theta_{t-1} + \eta \cdot \nabla L(x_t, \theta_{t-1})$

SAÍDA: sequência de alertas e parâmetros θ_t atualizados

PÓS-CONDIÇÃO: modelo reflete o comportamento recente do processo

Score de Detecção

A cada instante t , o algoritmo computa um score que mede o desvio da observação atual em relação à referência corrente. Se o score ultrapassa o limiar, uma mudança é sinalizada.

$$s_t = d(x_t, \mu_{t-1})$$

Alerta se $s_t > \tau$

Onde d é uma função de discrepância (ex: distância, log-verossimilhança) e τ controla o equilíbrio entre sensibilidade e falsos alarmes.

ENTRADA: x (fluxo de observações), τ (limiar de detecção)

PRÉ-CONDIÇÃO: referência μ_0 estimada com dados iniciais

PASSO 1 — Para cada nova observação x_t :

$s_t \leftarrow d(x_t, \mu_{t-1})$ calcula o score de desvio

PASSO 2 — Decisão

SE $s_t > \tau$:

emitir alerta no instante t

(opcional) reiniciar referência $\mu_t \leftarrow x_t$

PASSO 3 — Atualização da referência

$\mu_t \leftarrow \mu_{t-1} + (x_t - \mu_{t-1}) / t$

SAÍDA: instantes de alerta detectados

PÓS-CONDIÇÃO: referência μ_t reflete o histórico acumulado até t

Tipicidade Adaptativa

Na detecção online, o que é "normal" muda com o tempo. A distribuição de referência P_t evolui à medida que novos dados chegam — tornando a tipicidade um alvo móvel.

$$\text{typ}(x_t) = P_t(x_t)$$

Uma observação x_t é atípica se sua probabilidade sob a referência atual P_t for baixa — disparando um sinal de detecção.

A tensão central: o modelo deve se adaptar a mudanças legítimas, mas ainda sinalizar anomalias — em tempo real e com informação parcial.

Mudança Temporal: Change Points e Concept Drift

O livro distingue dois fenômenos centrais de mudança em séries temporais, cada um com paradigma e métodos próprios.

Change Points

Mudança na dinâmica da série (distribuição ou autocorrelação). Pode ser abrupta ou gradual. Detectada **offline** (série completa) ou **online** (tempo real).

Concept Drift

Monitoramento online de mudanças em séries em streaming: $p(\chi_i) \neq p(\chi_{i+1})$ entre lotes consecutivos. Classificado como **virtual** (mudança em X ou Y sem alterar o conceito) ou **real** (mudança no conceito χ).

Tipos de Drift

Abrupto — mudança brusca entre lotes.

Incremental — transição suave.

Gradual — alternância até estabilizar.

Recorrente — retorno periódico ao conceito anterior.

Distinguir esses fenômenos é essencial para escolher o método de detecção e a abordagem de tratamento adequados.

Métodos Online: Visão Comparativa

Os métodos de detecção diferem pela forma como definem tipicidade e identificam mudanças. A tabela resume as sete principais famílias — estatísticos sequenciais, probabilísticos, aprendizado online, distribuição, distância, árvore e projeção —, com exemplos de detectores representativos.

Família	Ideia Central	Quando Usar	Principal Limitação
Estatísticos Sequenciais <i>(CUSUM, EWMA, Page-Hinkley)</i>	Acumulam evidência estatística ao longo do tempo	Mudanças persistentes de nível ou média	Sensível ao limiar; assume estacionariedade prévia
Probabilísticos	Modelam a distribuição e detectam desvios de probabilidade	Dados com distribuição conhecida ou estimável	Custo de atualização do modelo
Aprendizado Online <i>(DDM, HDDM, ECDD)</i>	Modelo preditivo adaptativo; evento = erro de previsão elevado	Séries com padrões complexos e não-estacionários	Risco de adaptação ao evento (esquecimento)
Distribuição <i>(ADWIN, KSWIN, KLD, MCDD)</i>	Comparam distribuições entre lotes consecutivos	Detecção de virtual drift e mudanças em X	Requerem definição de tamanho de lote
Distância <i>(COD, CPOD)</i>	Monitoramento de distâncias entre observações no fluxo	Eventos raros e bem separados do histórico	Custo quadrático; dificuldade em alta dimensão
Árvore <i>(HS-Tree, RRCF)</i>	Particionamento hierárquico com atualização incremental	Streams multivariados não estacionários	Complexidade de implementação e ajuste de hiperparâmetros
Projeção <i>(LODA, XStream)</i>	Redução dimensional via projeções aleatórias	Streams de alta dimensão com restrição computacional	Perda de informação pela redução dimensional

Pipeline de Detecção Online

A detecção online pode ser vista como um pipeline modular onde cada etapa transforma incrementalmente os dados brutos em decisões sobre eventos temporais.

1. Ingestão de Dados

Observações x_t chegam sequencialmente em fluxo contínuo

2. Gerenciamento de Memória

Define o que reter: memória completa, sem memória ou janela deslizante

3. Atualização Incremental

Estatísticas e parâmetros atualizados a cada observação com custo $O(1)$

4. Cálculo do Score

Computa desvio S_t em relação à referência atual

5. Decisão

$S_t > \theta \rightarrow$ evento detectado e alarme emitido

6. Adaptação

Atualiza o modelo ou referência para lidar com concept drift

Cada família de métodos instancia esse pipeline de forma diferente, mas a estrutura modular é sempre a mesma.

Latência versus Precisão: o Trade-off Central

Detectar cedo reduz o atraso de resposta, mas aumenta o risco de falsos positivos. Esse é o trade-off fundamental de qualquer sistema de detecção online.

Velocidade vs. Acurácia

- **Baixa latência:** menor atraso $\Delta t = \hat{\tau} - \tau$, mas mais falsos positivos
- **Alta precisão:** menos alarmes falsos, mas o evento pode já ter causado impacto irreversível

Inadequação das Métricas Clássicas

- Precisão, recall e acurácia não capturam **quando** a detecção ocorreu, apenas **se** ocorreu
- Uma detecção correta com grande atraso pode ser tão problemática quanto uma detecção errada

Viés Temporal e Métricas Adequadas

- **Viés temporal:** tendência sistemática de antecipar ou atrasar detecções em relação ao evento real
- Detecções precoces geram alarmes falsos; detecções tardias impedem respostas a tempo

A calibração do limiar θ controla o trade-off velocidade–precisão, mas a avaliação rigorosa exige métricas que integrem acurácia e viés temporal.

Complexidade Computacional em Streaming

Em streaming, cada nova observação deve ser processada imediatamente — sem rever o histórico completo. O requisito central é duplo: custo por passo $O(1)$ e uso de memória limitado.

Métodos Batch

Reprocessam toda a série a cada nova observação. Custo $O(n)$ ou $O(n^2)$ por passo e memória irrestrita. Impraticável em tempo real.

Métodos Incrementais

Atualizam apenas estatísticas suficientes a cada nova observação. Custo $O(1)$ e memória constante. Escaláveis para fluxos contínuos de alto volume.

Janela Deslizante

Mantêm uma janela de tamanho fixo w . Custo $O(w)$ por passo. Reduzem o atraso de detecção sem exigir o histórico completo.

Alta Dimensionalidade

Métodos baseados em distância incorrem em custo quadrático. Abordagens baseadas em árvores ou projeções oferecem melhor escalabilidade para streams multivariados.

A escolha do método deve equilibrar acurácia de detecção, viabilidade de memória e custo computacional por observação.

Ponto Único de Quebra – AMOC

AMOC (**At Most One Change**) é o método fundamental de detecção de ponto de quebra. Encontra a única quebra estrutural mais significativa minimizando uma função de custo sobre todos os pontos de divisão possíveis.

Intuição Matemática

$$\hat{\tau} = \arg \min_{\tau} [C(x_{1:\tau}) + C(x_{\tau+1:n})]$$

O ponto de divisão ótimo $\hat{\tau}$ marca a localização da única quebra estrutural mais significativa.

Objetivo

Detectar o único ponto de mudança mais significativo, minimizando globalmente uma função de custo sobre todos os candidatos.

Premissas

A série contém **no máximo um ponto de quebra**; não adequado para dados com múltiplas mudanças estruturais.

Pontos Fortes

Simples, interpretável e **computacionalmente barato** — varredura $O(n)$; ideal como linha de base pedagógica.

Limitações

Falha com múltiplas quebras. **Sensível** à escolha da função de custo, que pode deslocar a localização do ponto de quebra.

 **Leitura Complementar:** Hinkley, D.V. (1970). Inference about the change-point in a sequence of random variables. *Biometrika*, 57(1), 1–17. — O artigo seminal que estabelece a base estatística para a inferência de um único ponto de mudança.

Detectar, Avaliar e Visualizar

```
# Executar detecção na série completa
detection <- detect(model, dataset$serie)
```

```
# Exibir índices detectados
print(detection |>
  dplyr::filter(event == TRUE))
```

```
## idx event type
## 1 389 TRUE changepoint
```

```
# Avaliar contra eventos rotulados
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
```

```
## event
## detection TRUE FALSE
##   TRUE  0   1
##  FALSE  4 495
```

```
# Plotar detecções sobre a série
har_plot(model, dataset$serie,
  detection, dataset$event)
```

Passo a passo

- `detect()` retorna um data frame com coluna `event` marcando índices detectados
- Use `dplyr::filter(event == TRUE)` para isolar os pontos de mudança
- O AMOC sinalizou o índice **389** como único ponto de mudança mais significativo
- `evaluate()` compara as detecções com os rótulos reais via matriz de confusão
- `har_plot()` sobrepõe detecções e verdade ao terreno na série

❏ **Conceito-chave:** Interprete os resultados como evidência da adequação do método aos dados, não como números abstratos.

Múltiplos Pontos de Quebra – Segmentação Binária (BinSeg)

BinSeg é um **algoritmo recursivo guloso** que estende o AMOC para detectar múltiplas mudanças estruturais, dividindo segmentos recursivamente até atingir um número máximo de pontos de quebra **Q** ou um limiar de penalidade.

Intuição Matemática

A cada etapa, encontre o ponto de divisão ótimo minimizando o custo total entre os dois segmentos:

$$\tau^* = \arg \min_{\tau} [\mathcal{C}(x_{1:\tau}) + \mathcal{C}(x_{\tau+1:n})]$$

Divida em τ^* e repita em cada subsegmento até o critério de parada ser satisfeito.

✓ Pontos Fortes

- **Rápido:** $O(n \log n)$ — escala bem para séries longas
- **Amplamente utilizado** em diversas áreas
- **Fácil de implementar;** baseia-se diretamente no AMOC

⚠ Limitações

- **Guloso:** divisões anteriores nunca são revisitadas
- **Não é globalmente ótimo:** sem garantia da melhor segmentação
- **Sensível a Q:** o número máximo de quebras influencia fortemente os resultados

📖 **Leitura Complementar:** Vostrikova, L.Yu. (1981). Detecção de "desordem" em processos aleatórios multidimensionais. *Soviet Mathematics Doklady*, 24, 55–59.

Múltiplos Pontos de Quebra – Particionamento Ótimo (PELT)

PELT é um algoritmo exato que encontra a segmentação globalmente ótima de uma série temporal sob um framework de custo penalizado. Ao contrário do BinSeg, garante o conjunto globalmente ótimo de pontos de quebra com tempo de execução quase linear via poda.

Intuição Matemática

PELT minimiza o custo total penalizado sobre todas as segmentações possíveis:

$$\text{minimizar sobre todo } \tau : \sum \mathcal{C}(x_{\tau_{i-1}+1:\tau_i}) + \beta \cdot m$$

onde $\mathcal{C}$ é o custo dentro do segmento, β é a penalidade por ponto de quebra e m é o número de pontos de quebra. A regra de poda elimina candidatos τ sempre que:

$$\mathcal{C}(x_{\tau:t}) + \beta \geq \mathcal{C}(x_{\tau:s}) + \mathcal{C}(x_{s:t}) \quad \text{para todo } t \text{ futuro}$$

Isso garante tempo de execução linear esperado sob condições distribucionais suaves.

✓ Pontos Fortes

- Segmentação globalmente ótima (não gulosa)
- Tempo esperado quase linear $O(n)$
- Penalidade β controla complexidade — β maior $\rightarrow$ menos pontos de quebra

⚠ Limitações

- Otimalidade depende da escolha de β — valor inadequado causa sobre- ou subdetecção
- Assume função de custo apropriada para a distribuição dos dados

Leitura Adicional: Killick, R., Fearnhead, P., & Eckley, I.A. (2012). Detecção ótima de pontos de mudança com custo computacional linear. *Journal of the American Statistical Association*, 107(500), 1590–1598.

Resumo: Famílias de Métodos e Quando Usá-los

Clássico – AMOC

`hcp_amoc()` — Use quando se espera **no máximo uma** quebra estrutural. Simples, rápido e interpretável.

Clássico – PELT

`hcp_pelt()` — Use quando a segmentação **globalmente ótima** é necessária. Minimização de custo penalizado; ajuste β para controlar sensibilidade.

ChangeFinder – LR, ARIMA, ETS

`hcp_cf_lr()` / `hcp_cf_arima()` / `hcp_cf_ets()` — Use quando mudanças se manifestam como **picos no erro de previsão**. Detecta surpresas preditivas.

Clássico – BinSeg

`hcp_binseg(Q)` — Use com **múltiplos breakpoints** e velocidade é prioridade. Divisões recursivas gulosas; Q define o limite superior.

Testes Estatísticos – Chow & GFT

`hcp_chow()` / `hcp_gft()` — Use quando se prefere um **framework de teste de hipóteses**. Chow testa um ponto candidato; GFT varre todas as localizações via F-estatísticas.

Volatilidade & Janelas – GARCH & SCP

`hcp_garch()` / `hcp_scp()` — Use para mudanças na **dinâmica de variância** (GARCH) ou detecção por **contraste local de janelas** (SCP).

Testes de Flutuação Generalizada (GFT)

```
# Dataset simples
dataset <- examples_changepoints$simple

# Configurar e ajustar GFT
model <- hcp_gft()
model <- fit(model, dataset$serie)

# Executar detecção
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## idx event type
## 1 49 TRUE changepoint
## 2 64 TRUE changepoint
```

```
# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)

## event
## detection TRUE FALSE
##   TRUE  0   2
##   FALSE 1  98

# Plotar
har_plot(model, dataset$serie,
  detection, dataset$event)
```

Passo a Passo

- `hcp_gft()` utiliza `strucchange::breakpoints()` internamente para detecção
- Avalia a estabilidade dos parâmetros de regressão ao longo do tempo, sinalizando instabilidades estruturais
- Candidatos a breakpoint são selecionados por critérios de informação (BIC/AIC)
- Índices detectados **49 e 64** — ambos foram falsos positivos neste dataset
- O GFT tende a ser mais sensível que o Chow, propenso a sobre-detecção em séries com flutuações graduais

📌 **Conceito-chave:** O GFT avalia a estabilidade dos parâmetros de regressão ao longo do tempo — útil quando a quebra estrutural é gradual, não abrupta.

ChangeFinder com Regressão Linear

```
# Dataset simples
dataset <- examples_changepoints$simple

# Configurar CF-LR
# sw_size controla o tamanho da janela
model <- hcp_cf_lr(sw_size = 10)
model <- fit(model, dataset$serie)

# Detectar
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## [1] idx event type
## <0 rows> (or 0-length row.names)
```

```
# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
## detection TRUE FALSE
##   TRUE  0  0
##   FALSE 1 100

# Plotar série e resíduos
har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection, "threshold"))
```

Passo a Passo

- `sw_size = 10` define o tamanho da janela deslizante usada para suavização
- O modelo rastreia desvios dos resíduos e aplica suavização em dois estágios
- Nenhum evento detectado — 0 verdadeiros positivos e 0 falsos positivos nesta execução
- O segundo `har_plot()` exibe as pontuações dos resíduos junto ao limiar de decisão
- Use `attr(detection, "res")` e `attr(detection, "threshold")` para acessar esses valores diretamente

📌 **Conceito-chave:** O ChangeFinder aplica suavização em dois estágios sobre os resíduos do modelo — o limiar de detecção é aprendido automaticamente via `harutils()`, sem necessidade de definição manual.

ChangeFinder com ARIMA

```
# Dataset simples
dataset <- examples_changepoints$simple

# Configurar CF-ARIMA
model <- hcp_cf_arima(sw_size = 10)
model <- fit(model, dataset$serie)

# Detectar
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## idx event type
## 1 51 TRUE anomaly
```

```
# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
## detection TRUE FALSE
##   TRUE  0   1
##  FALSE  1  99

# Plotar série e resíduos
har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection, "threshold"))
```

Passo a Passo

- `hcp_cf_arima(sw_size = 10)` usa ARIMA internamente via pacote `forecast`
- O modelo sinalizou o índice **51** como **anomaly** — não como **changepoint**
- Resultado: 0 verdadeiros positivos e 1 falso positivo — o evento real foi ignorado
- O gráfico de resíduos mostra onde o score ultrapassou o limiar aprendido
- Compare o gráfico de resíduos com o da série para entender o comportamento do modelo

📌 **Conceito-chave:** O ARIMA captura autocorrelação temporal nos resíduos — sendo mais adequado para séries com forte dependência temporal do que a regressão linear simples.

ChangeFinder com GARCH

```
# Dataset simples
dataset <- examples_changepoints$simple

# Configurar GARCH-based ChangeFinder
model <- hcp_garch()
model <- fit(model, dataset$serie)

# Detectar
detection <- detect(model, dataset$serie)
print(detection |>
  dplyr::filter(event == TRUE))

## idx event type
## 1 50 TRUE changepoint
```

```
# Avaliar
evaluation <- evaluate(model,
  detection$event, dataset$event)
## detection TRUE FALSE
##   TRUE  1   0
##   FALSE 0 100

# Plotar série e resíduos
har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection, "threshold"))
```

Passo a passo

- `hcp_garch()` modela a variância condicional da série temporal
- Os escores residuais são suavizados e limiarizados para detectar mudanças de volatilidade
- Identificou exatamente o índice **50** — resultado perfeito: **1 VP, 0 FP**
- O gráfico de resíduos revela claramente o pico de variância no ponto de mudança
- GARCH é especialmente eficaz em séries financeiras com agrupamento de volatilidade

📌 **Conceito-chave:** GARCH captura mudanças de regime na *variância*, não apenas na média — sendo ideal quando a instabilidade se manifesta como aumento de volatilidade, e não como mudança de nível.

Resumo dos Métodos

Visão comparativa de todos os detectores de pontos de mudança disponíveis no Harbinger, organizados por família e caso de uso.

Método	Função	Abordagem	Ideal para
AMOC	hcp_amoc()	Otimização de custo — único ponto	Um ponto de mudança dominante
BinSeg	hcp_binseg(Q)	Segmentação binária recursiva	Múltiplos pontos, rápido
PELT	hcp_pelt()	Particionamento ótimo com poda	Múltiplos pontos, ótimo
Chow	hcp_chow()	Estatística F em regressão linear	Quebras em modelos lineares
GFT	hcp_gft()	Flutuação de parâmetros de regressão	Instabilidade estrutural gradual
CF-LR	hcp_cf_lr(sw)	Resíduos de regressão linear	Surpresa preditiva simples
CF-ARIMA	hcp_cf_arima(sw)	Resíduos ARIMA suavizados	Séries com autocorrelação
CF-ETS	hcp_cf_ets(sw)	Resíduos ETS suavizados	Tendência e sazonalidade
GARCH	hcp_garch()	Modelagem de variância condicional	Mudanças de volatilidade
SCP	hcp_scp(sw)	Contraste em janela local	Quebras localizadas e abruptas

O que cobrimos na Parte 6

Razão de Verossimilhança:
compara dois modelos para
detectar mudança
estrutural via estatística
 $\Lambda(\tau)$

**Programação Dinâmica
(PELT):** particionamento
globalmente ótimo
minimizando custo
penalizado pelo número de
cortes

Change Points via HMM:
alternância entre K regimes
ocultos detectada por
transições de estado
latente z_t

**Change Points via
Aprendizado de Máquina:**
erro de previsão agregado
em janela como sinal de
mudança de regime

**Paradigmas offline vs.
online:** acesso a
observações futuras define
o modo de operação do
detector

Latência de detecção:
atraso entre ocorrência do
evento e sinalização –
trade-off central do
paradigma online

**Janela deslizante e
estatísticas incrementais:**
atualização eficiente de
estimativas sem recalcular
do zero

**Modelos adaptativos e
score de detecção:**
parâmetros ajustados
incrementalmente; limiar
sobre score sinaliza
mudança

Famílias de métodos: AMOC, BinSeg, PELT, Chow, GFT, CF-LR, CF-ARIMA, CF-ETS, GARCH e SCP – cada um com seu nicho de aplicação